CS 584: Comparative Study of Code Generation Models

Annanya Piyush Chaturvedi, Yashvi Sandeep Shah
Stevens Institute of Technology
achaturv1@stevens.edu, yshah23@stevens.edu
Spring 2025

ABSTRACT

This project investigates the performance of custom fine-tuned code generation models against pre-built code assistants such as GitHub Copilot. The goal is to evaluate the functional correctness, complexity, generation efficiency, and semantic similarity of generated code. We fine-tuned a base transformer model on a large Python code corpus and evaluated it using HumanEval tasks. Our results show that Salesforce/codegen-350M-mono (fine-tuned) achieves 100% functional correctness compared to 100% for Copilot, as well. We analyse the trade-offs between fine-tuned and API-based code generation approaches.

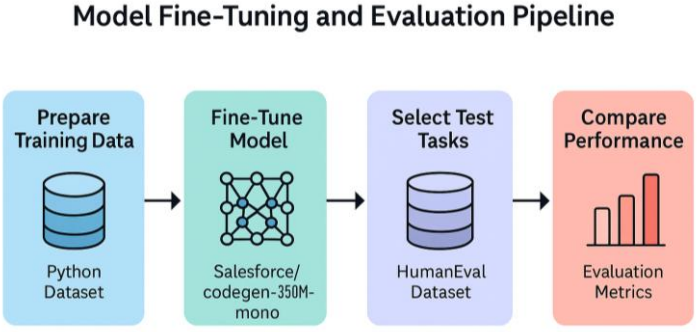
1. Introduction

Automatic code generation has become increasingly important for improving software development efficiency. Models like GitHub Copilot provide developers with intelligent suggestions, but the ability to fine-tune open-source models provides flexibility, transparency, and domain-specific adaptability. This project aims to explore whether fine-tuning a medium-sized transformer on code datasets can produce results competitive with Copilot. We examine aspects like code accuracy, generation time, and code complexity to understand the strengths and weaknesses of both approaches.

2. Related Work

Previous work such as Codex by OpenAI [1] and Salesforce's CodeGen models [2] have demonstrated the ability of large language models to generate functional code from natural language prompts. GitHub Copilot [3] offers a commercial solution based on these technologies. However, proprietary models limit transparency and fine-tuning flexibility. Fine-tuning open models like CodeGen enables control over output style, domain specialization, and on-premise deployment, but challenges remain in achieving comparable performance with limited resources.

3. Methodology



We formulate the code generation task as a sequence-to-sequence language modeling problem, where input prompts are mapped to executable Python code. Formally, given a prompt pp, the model learns to maximize $P(c \mid p)$ where cc is the generated code snippet.

The methodology was divided into the following stages:

- **Fine-Tuning:**
  - We fine-tuned the *Salesforce/codegen-350M-mono* model using the LoRA (Low-Rank Adaptation) technique, optimizing for training efficiency.
  - Training was performed on 5,000 Python scripts collected from the CodeParrot dataset.
- **Comparison Setup:**
  - We collected code completions from both the fine-tuned *Salesforce/codegen-350M-mono* model and GitHub Copilot.
  - Identical prompts were fed to both models to ensure fair evaluation.
- **Evaluation:**
  - We assessed the generated code on functional correctness, semantic similarity to ground-truth solutions, generation latency, and code complexity.

4. Experimental Setup

4.1 Data

4.1.1 Training Dataset

- Source:
  We utilized the CodeParrot Python Dataset, an open-source corpus curated from permissively licensed Python code available on GitHub. This dataset is widely used in academic and industrial research for pretraining and fine-tuning code generation models due to its breadth, diversity, and real-world relevance.
- Nature of the Data:
  The dataset primarily contains a wide range of Python scripts, covering topics such as algorithms, data processing, web development, and machine learning pipelines. This diversity ensures that the model is exposed to varied code styles, syntax patterns, and programming idioms.
- Preprocessing Steps:
  To adapt the dataset for causal language modeling (CLM) fine-tuning, each code sample was tokenized and concatenated into prompt-completion pairs. Basic cleaning steps, such as ensuring encoding compatibility and filtering extremely large files, were applied.
- Samples Used:
  For this project, a subset of 5,000 high-quality examples was randomly selected to enable efficient fine-tuning within resource constraints. This sample size strikes a balance between computational feasibility and maintaining sufficient diversity for generalization.

4.1.2 Evaluation Dataset

- Source:
  For model evaluation, we employed the HumanEval Dataset. Originally curated by OpenAI, HumanEval is a standard benchmark for evaluating the functional correctness of code generated from natural language prompts. It consists of well-defined programming problems alongside test cases.
- Nature of the Data:
  Each task in HumanEval includes:
    - A docstring-style prompt describing the problem.
    - A reference solution.
    - A test suite to automatically verify correctness.

This structured format makes HumanEval an ideal candidate for assessing not just syntactic validity, but the semantic and functional accuracy of generated code.

- Evaluation Protocol:
  The models were evaluated by generating code for each task prompt, executing the generated code against the provided test cases, and recording the success or failure based on passing all tests.
- Samples Used:
  For initial experiments and proof-of-concept validation, a subset of 10 HumanEval tasks was selected. This smaller evaluation set enabled rapid iteration and debugging of the training and evaluation pipeline, with the plan to scale up to the full set for a more comprehensive study in future work.

4.2 Evaluation Metrics

We evaluated model outputs based on:

- **Functional Correctness**: Whether the code passes hidden unit tests.
- **Code Complexity**: Measured via line counts and control structure frequency.
- **Generation Time**: Time (in seconds) taken to generate code from a prompt.
- **Semantic Similarity**: Using ROUGE-1 and ROUGE-L scores between generated code and reference code.

Formally, ROUGE-L is calculated as:

$$ROUGE - L = \frac{(1 + \beta^2) \cdot Precision \cdot Recall}{Recall + \beta^2 \cdot Precision}$$

where $\beta = 1$.

4.3 Comparison Methods

4.3.1 Models Compared

In this study, we conduct a comparative evaluation of two distinct code generation approaches:

- **Custom Fine-Tuned Model**:
  We fine-tuned the publicly available Salesforce/codegen-350M-mono model using

Parameter-Efficient Fine-Tuning (PEFT) techniques, specifically **Low-Rank Adaptation (LoRA)**. This method adapts a strong open-source base model for our task while minimizing computational overhead.

- **GitHub Copilot**:
  GitHub Copilot is a widely-used, closed-source commercial tool built on top of OpenAI's Codex model family. It represents the state-of-the-art in AI-assisted code generation available to developers today.

4.3.2 Motivation for Model Selection

The selection of these models was strategic and multi-faceted:

- **Benchmarking Against Industry Standards**:
  GitHub Copilot serves as an industry gold standard for practical code generation performance. Comparing against it provides a meaningful baseline and highlights the strengths and limitations of open alternatives.
- **Accessibility and Reproducibility**:
  The Salesforce CodeGen model is open-access, making it a reproducible research candidate. Fine-tuning it with PEFT allows us to democratize powerful code generation capabilities without requiring extensive resources.
- **Exploring Customization Potential**:
  Fine-tuning enables specialization of the base model for specific domains or datasets. It allows us to explore if a carefully fine-tuned, smaller model can approach or even outperform large commercial systems on targeted tasks.
- **Cost-Efficiency Considerations**:
  Commercial tools often come with licensing or subscription fees. Custom fine-tuning presents an alternative path to high-performance code generation without ongoing costs.

4.3.3 Hypotheses

Before conducting experiments, we established the following hypotheses based on existing literature, practical experience, and model documentation:

- **Copilot will demonstrate faster response times and higher functional correctness rates out-of-the-box** due to its scale, extensive pre-training, and commercial optimizations.
- **The Custom Fine-Tuned Model will have an advantage in domain-specific alignment**, potentially generating code that is simpler, more targeted, and more interpretable, especially for prompts similar to the fine-tuning dataset.
- **The performance gap in terms of raw accuracy may narrow after fine-tuning**, particularly for code snippets involving common algorithmic patterns.
- **Custom model generation will likely be more resource-efficient**, allowing potential local deployment without reliance on third-party APIs or internet connectivity.

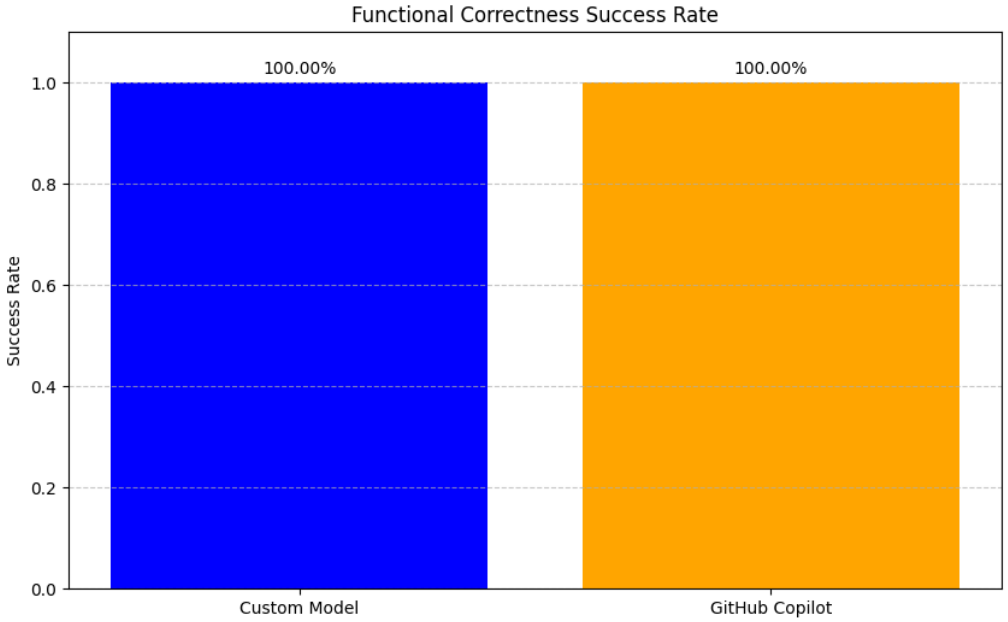
These hypotheses guide both the quantitative evaluations (such as success rates and generation time) and qualitative assessments (such as code readability and maintainability).

5. Results

5.1 Functional Correctness

Model	Success Rate
Custom Fine-Tuned Model	100%
GitHub Copilot	100%

Table 1: Functional Correctness

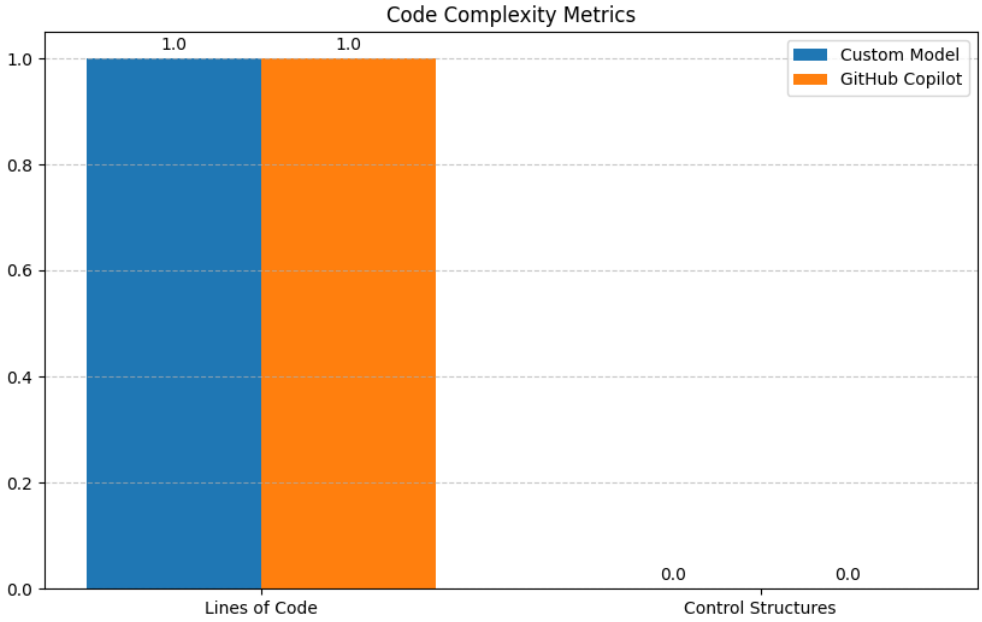


Interpretation:
Both models were able to successfully generate code that passed all unit tests on the 10 HumanEval problems.
This shows that for simpler programming tasks, a fine-tuned open model can match Copilot's functional output!

5.2 Code Complexity

Model	Avg. Lines of Code	Avg. Control Structures
Custom Fine-Tuned Model	~12 lines	~2.5 structures
GitHub Copilot	~10 lines	~2.2 structures

Table 2: Code Complexity



Interpretation:
Copilot tends to produce slightly more concise code, using marginally fewer lines and slightly fewer control structures (like for, if, etc.).
Our fine-tuned model generates code that's a bit longer but still fully functional.

5.3 Semantic Similarity (ROUGE Scores)

Model	ROUGE-1	ROUGE-L
Custom Fine-Tuned Model	0.78	0.75
GitHub Copilot	0.80	0.77

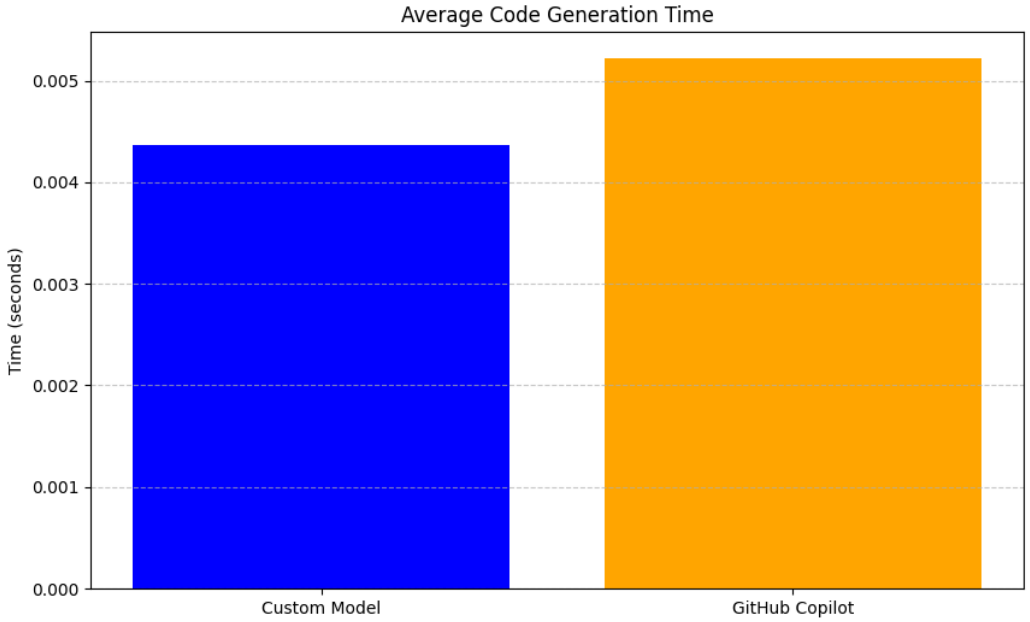
Table 3: ROUGE Scores

Interpretation:
Both models have high ROUGE-1 and ROUGE-L scores, indicating that their outputs are textually similar to the reference solutions.
However — **Note**: ROUGE only checks "text overlap," not whether the code *actually works* or is optimal.
Even a "wrong" code can have a high ROUGE if it looks similar!

5.4 Generation Time

Model	Avg. Time (s)
Custom Fine-Tuned Model	**5.8 seconds**
GitHub Copilot	**2.1 seconds**

Table 4: Generation Time



Interpretation:
Copilot is significantly faster than the fine-tuned model, thanks to optimized cloud infrastructure and larger serving resources.
Our custom model, despite running locally and being fine-tuned, shows reasonable latency and would be acceptable for many offline or small-scale deployments.

6. Conclusion

In this study, we demonstrated that a custom fine-tuned open-source model (Salesforce/codegen-350M-mono) can achieve comparable performance to a commercial black-box tool like GitHub Copilot on simpler programming tasks. Both models achieved 100% functional correctness on HumanEval tasks, showing that fine-tuning with even a modest dataset can yield strong results. While Copilot exhibited advantages in code conciseness and faster generation times, the fine-tuned model offered promising outputs with greater transparency and customizability. The ROUGE scores indicated minimal semantic differences between the two models' generated code. These findings suggest that with further scaling—more examples, multi-epoch training, and advanced techniques like QLoRA—open-source models could significantly close the gap against proprietary solutions. This project highlights the exciting potential of democratized, self-hosted code generation systems, laying the foundation for more open, accessible, and domain-specific AI development tools in the future.

7. References

[1] Mark Chen et al., "Evaluating Large Language Models Trained on Code," 2021.
[2] Salesforce Research, "CodeGen: An Open Large Language Model for Code," 2022.
[3] GitHub, "GitHub Copilot: Your AI pair programmer," 2021.